HARNESS LAB · MÓDULO 4 · AULA 4.4

E2E mínimo: *um fluxo, de ponta a ponta*

O critério que só o navegador vê. Um teste que usa a feature como o usuário usa, rodando no comando da Prova, o mesmo que o loop roda. Um fluxo, não uma suíte de E2E.

`Pest browser` `Playwright Test` `pytest-playwright` `HTTP test` `assertNoSmoke()` `->todo()` `webServer` `confere-e2e.py`

VOCÊ SAI COM

Um E2E do fluxo principal da sua feature-alvo, com o ID do critério no nome e pendente se a feature é nova, mais um smoke ativo que prova que o comando da Prova abre um navegador. O `confere-e2e.py` diz se os dois entram na Prova.

Material de apoio · Lucas Souza · Beer and Code

1 A aula em *uma página*

A aula arremata a parte de testes. O teste **end-to-end** (E2E) engloba tudo: o front, o back, o processo inteiro. Começa num ponto e vai até o fim, testando o fluxo como um todo. O unitário e o de feature dizem se cada peça está bem escrita; o E2E ainda pode falhar com todos eles verdes. É o teste do caminho feliz e do não feliz que o usuário faz na tela.



SUA STACK	E2E MÍNIMO	INSTALA COM
Laravel com tela	Pest browser (Pest 4+, PHP 8.3+)	`composer require pestphp/pest-plugin-browser --dev`
Laravel só API	HTTP test do Laravel	já vem no framework
Node / TS	Playwright Test	`npm init playwright@latest`
Python	pytest-playwright	`pip install pytest-playwright`

No Laravel, o exemplo foi o AC-03 da 4.3 clicando: visita a rota de cadastro, preenche os campos com um e-mail que já existe e confere a mensagem "esse e-mail já está cadastrado". Um teste só cobre do backend que valida até a mensagem que chega ao usuário. Sem tela, o E2E de uma API é chamar a rota e ver o ciclo completo. Em Node e Python, o mesmo teste com Playwright.

O que o mínimo cobre

O fluxo principal de uma feature, o serviço de fora fakeado (como na 4.2) e a saída indo para o gate da Prova, que o Módulo 5 constrói. Vários navegadores, CI, page objects e trace viewer ficam fora do mínimo.

O ambiente precisa estar pronto

Se você usa container, o navegador tem que estar instalado dentro dele. Configure você, ou ao menos peça ao seu assistente de código que configure o ambiente para rodar E2E (o pedido está no passo 5).

> *"These methods do not actually issue a "real" HTTP request to your application. Instead, the entire network request is simulated internally."*
>
> Documentação do Laravel 13.x · HTTP Tests, "Making Requests" (laravel.com/framework/docs/http-tests). O E2E sem tela vê a resposta e o banco, não o JavaScript.

Com isso o Módulo 4 fecha o arcabouço da avaliação do agente: definição de pronto, especificação estruturada (no curso, o formato GEARS, uma extensão do EARS), critérios bem definidos, teste por critério, um juiz de fora avaliando o que foi construído e o E2E para ver o todo trabalhando junto. O próximo passo, no Módulo 5, é automatizar a construção: o trilho, as travas e a avaliação já estão no lugar.

2 Indo *além da aula*

ALÉM DA AULA · TESTE PENDENTE NÃO ABRE O NAVEGADOR

O E2E da feature nasce pendente. Com a pasta de navegadores vazia, o E2E com `->todo()` passou ("1 todo") e a Prova ficou verde sem Chromium nenhum. O smoke ativo, na mesma situação, falhou: no Pest, `Playwright is outdated`; no Playwright Test e no pytest-playwright, `Executable doesn't exist`. Por isso o smoke fica ativo desde o primeiro dia: é ele que prova que o comando da Prova abre um navegador. O Pest fala em versão porque "Each version of Playwright needs specific versions of browser binaries to operate".

Fonte: playwright.dev/docs/browsers, "Introduction". Testado com Pest 5.3.5 e pest-plugin-browser 5.2.0, Playwright Test 1.64.1 (Node 22.20.0) e pytest-playwright 0.10.0.

ALÉM DA AULA · PROVA VERDE SEM TER RODADO O E2E

O starter kit do Laravel declara só `Unit` e `Feature` no `phpunit.xml`. Com o E2E em `tests/Browser` fora das `<testsuite>`, a suíte inteira passou (34 testes) sem rodar o E2E, que estava sem marcador e falharia. Só o `--filter` denuncia: "No tests found.". E se o `->in()` do `tests/Pest.php` não lista `Browser`, o teste roda sem o Laravel e cai em `Target class [config] does not exist.` antes de abrir a página. No Python, o loop só acha o pytest com `pytest.ini` ou `[tool.pytest]` no `pyproject.toml`. O `confere-e2e.py` olha os três.

Fonte: teste num projeto Laravel 13 com Pest 5.3.5; pestphp.com/docs/browser-testing (o exemplo "may sign in the user" assume o RefreshDatabase no Pest.php); ordem de detecção do `qual-comando.sh` da 4.1.

ALÉM DA AULA · VITEST E PLAYWRIGHT BRIGAM PELOS MESMOS ARQUIVOS

O `"test": "vitest run && playwright test"` do slide só funciona com cada um na sua pasta. O padrão do Vitest pega todo `*.spec.ts`: rodou o arquivo do Playwright e quebrou com `test.skip() can only be called inside test, describe block or fixture`. Do outro lado, `testDir: './tests'` no Playwright pegou o teste do Vitest: `Vitest failed to find the runner`. A saída é o `exclude` da pasta do E2E no Vitest e o `testDir` na pasta do E2E, como nos modelos. E o Playwright pede "Node.js: latest 22.x, 24.x or 26.x".

Fonte: testado com Vitest 4.1.12 e Playwright Test 1.64.1; playwright.dev/docs/intro, "System requirements".

ALÉM DA AULA · O NAVEGADOR MORA ONDE A PROVA RODA

Com `laravel/sail` no `composer.json` (o starter kit traz), o loop escolhe `vendor/bin/sail test` mesmo que você não use Sail: aí, com os containers parados, o loop para antes de começar, e a saída é o `LOOP_TEST_CMD` na frente do `loop.sh` (aula 4.1; no ralph puro, `--test-cmd`). Se usa Sail, o teste roda dentro do container. A doc do Sail cobre o Dusk, com um container Selenium; Pest browser dentro do Sail não está na doc do Pest e não foi testado aqui. Num Linux limpo, o Chromium precisa das bibliotecas do sistema: `npx playwright install --with-deps chromium` instala "the browsers and OS dependencies" (sem root, ele tenta virar root). A imagem oficial do Playwright para Docker traz navegadores e dependências, mas não o pacote. E o que você instala à mão num container de pé some quando ele é recriado: vai no Dockerfile ou no compose.

Fonte: playwright.dev/docs/browsers, "Install system dependencies"; playwright.dev/docs/docker; laravel.com/framework/docs/sail, "Laravel Dusk"; `qual-comando.sh` da 4.1 (exit 5, containers parados). A prova é o smoke com `vendor/bin/sail test --filter=Smoke`.

ALÉM DA AULA · SERVIÇO DE FORA FAKEADO: ONDE O DUBLÊ VIVE

No Pest browser, o teste e o app estão no mesmo Laravel: o exemplo da doc chama `Event::fake()` antes do `visit()` e confere o evento no fim, com "the full power of Laravel's testing capabilities"; a doc do Pest Agent faz o mesmo com `Mail::fake()`. No Playwright Test, o app roda em outro processo: mock dentro do teste não chega ao servidor, e o `page.route()` só troca o que a página pede ("Any requests that a page does"). A chamada que o servidor faz para fora é trocada pela configuração do app no teste: o `webServer` aceita `env`, que herda o ambiente "with `PLAYWRIGHT_TEST=1` added". Esse `env` é só do Node. No Python, quem sobe o app é uma fixture do `conftest.py`, e o ambiente vai nela; no Django, o `live_server` roda numa thread do próprio pytest, e o mock do teste chega ao app.

Fonte: pestphp.com/docs/browser-testing e /docs/agent ("Combining Frontend and Backend"); playwright.dev/docs/mock e /docs/test-webserver; pytest-django, "live_server". Testado com pytest-playwright 0.10.0, Django 6.1.3 e pytest-django 4.14.0.

ALÉM DA AULA · /VERIFY E --CHROME AJUDAM A VER, NÃO JULGAM

O Claude Code também abre navegador: `--chrome` liga a integração com o Chrome "for web automation and testing", e o `/verify` sobe o app "to confirm a code change does what it should, without falling back to tests". Bons para olhar a tela durante a sessão, mas quem decide ali é o modelo. Na tabela da doc, "A request for evidence" fica do lado não determinístico; "Tests, typecheck, a Stop hook, CI", do determinístico. O E2E no comando da Prova roda fora da sessão, sempre igual. No outro extremo, o agente "healer" do Playwright "automatically repairs failing tests": o oposto da 4.2, em que o agente não reescreve o teste de aceite.

Fonte: code.claude.com/docs/en/cli-reference; /skills, "Run and verify your app"; /best-practices, "Deterministic and non-deterministic instructions"; playwright.dev/docs/test-agents.

3 Instale e faça *no seu repositório*

Antes: o `FEATURE-ALVO.md` com os critérios e os testes de aceite da 4.3 commitados, e o comando da Prova da 4.1 verde. Escolha **uma** linha da tabela da seção 1.

1 Baixe os materiais do curso (uma vez só).

Terminal · na sua pasta pessoal — shell

```
cd ~
git clone https://github.com/beerandcodeteam/harness-lab-materiais
```

Já baixou antes? Atualize:

Terminal · em qualquer pasta — shell

```
git -C ~/harness-lab-materiais pull
```

2 Instale o conferidor da aula.

Cria `.claude/scripts/confere-e2e.py` (precisa de `python3`). Ele acha os E2E, descobre o comando da Prova na ordem do `qual-comando.sh` da 4.1 e diz se cada E2E entra nele. Só lê arquivos, nunca o `.env`.

Terminal · na raiz do seu projeto — shell

```
~/harness-lab-materiais/instalar.sh 4.4
```

3 Instale a ferramenta de E2E da sua stack.

Laravel só API: nada a instalar. Laravel com tela, da doc do Pest; com Sail, rode no container (`vendor/bin/sail composer`, `... npm`, `... npx`). Container ou Linux limpo: veja a seção 2.

Terminal · projeto Laravel — laravel

```
composer require pestphp/pest-plugin-browser --dev
npm install playwright@latest
npx playwright install
```

O plugin só existe no Pest 4 (PHP 8.3+) e no Pest 5 (PHP 8.4+): com Pest 3 ou PHP mais antigo, o Composer recusa. Aí, ou com PHPUnit puro, o E2E é o HTTP test (`modelos/laravel-http/`), sem tela, ou o Laravel Dusk, com o comando `dusk` no `LOOP_TEST_CMD` do `loop.sh` (no `confere-e2e.py`, por `--test-cmd`, e ele reconhece).

Terminal · projeto Node — node

```
npm init playwright@latest
```

Terminal · projeto Python — python

```
pip install pytest-playwright
playwright install
```

O `npm init` cria um config que roda os testes "across Chromium, Firefox and WebKit" e um `tests/example.spec.ts` que abre playwright.dev. Apague o example e use o

`modelos/playwright/playwright.config.ts`: um navegador, `testDir` e `webServer`.

4 **Ponha o smoke da sua stack, ativo.**

Um teste sem marcador que abre a home e confere erro de JavaScript. No Laravel, o bloco também põe as duas pastas do Pest no `.gitignore`.

Terminal · na raiz do seu projeto Laravel — shell

```
A=~/harness-lab-materiais/modulo-4-prova/aula-4.4-e2e-minimo/modelos
mkdir -p tests/Browser
cp "$A/pest-browser/SmokeTest.php" tests/Browser/
cat "$A/pest-browser/gitignore.exemplo" >> .gitignore
```

Terminal · na raiz do seu projeto Node — shell

```
A=~/harness-lab-materiais/modulo-4-prova/aula-4.4-e2e-minimo/modelos
mkdir -p tests/e2e
cp "$A/playwright/smoke.spec.ts" tests/e2e/
```

Terminal · na raiz do seu projeto Python — shell

```
A=~/harness-lab-materiais/modulo-4-prova/aula-4.4-e2e-minimo/modelos
mkdir -p tests/e2e
cp "$A/pytest-playwright/test_smoke_e2e.py" tests/e2e/
```

5 **Peça ao Claude o resto do ambiente.**

É a sugestão da aula: pedir ao assistente que configure o ambiente para rodar E2E. Cole no Claude Code. A versão completa, com o que conferir na resposta, é `modelos/pedir-o-ambiente.md`.

```
Configure este projeto para rodar testes E2E no comando de teste que o loop roda (a Prova).
Descubra qual é esse comando e onde ele roda (minha máquina, container, CI) e me diga antes de
mudar. A ferramenta de E2E e o Chromium ficam no mesmo ambiente da Prova; se for container, no
Dockerfile ou no compose. A Prova precisa achar o teste: no Laravel, tests/Browser numa
<testsuite> do phpunit.xml e no ->in() do tests/Pest.php; no Node, o script test chama o
playwright test, o Vitest não pega a pasta do E2E e o config tem um navegador só (sem Firefox,
WebKit e o example.spec.ts do npm init); no Python, testpaths no pytest.ini e o próprio pytest
sobe o app (Django: base_url vindo do live_server do pytest-django; FastAPI ou Flask: fixture
de sessão no conftest.py que sobe o servidor e espera a porta). O smoke fica ativo; não escreva
o E2E da feature agora e não mexa em teste que já existe. No fim, rode o comando da Prova
inteiro, me mostre o comando e a saída, e rode python3 .claude/scripts/confere-e2e.py até não
sobrar ERRO.
```

6 **Confira: o smoke entra na Prova e passa nela.**

Lista cada E2E (ativo ou pendente) e uma linha ok, AVISO ou ERRO por checagem. Tem que terminar com "os E2E entram no comando do juiz" (saída 0).

Terminal · na raiz do seu projeto — shell

```
python3 .claude/scripts/confere-e2e.py
```

Depois, rode o smoke no comando da Prova, no ambiente dela (Node e Python: com o app fechado, porque quem sobe o app é o teste). Laravel com Sail, abaixo; sem Sail, `./vendor/bin/pest --filter=Smoke`; Node, `npm test`; Python, `pytest`. O smoke tem que aparecer como passou.

Terminal · projeto Laravel com Sail — laravel

```
vendor/bin/sail test --filter=Smoke
```

7 **Peça o E2E do fluxo principal.**

Um pedido, um fluxo. Troque "Pest browser" por "Playwright Test", "pytest-playwright" ou "HTTP test do Laravel". O pedido do exemplo da aula e o que ler no resultado estão em `modelos/pedir-o-e2e.md`.

```
Escreva UM teste E2E com Pest browser para o fluxo principal da feature do FEATURE-ALVO.md: o caminho que o usuário faz do começo ao fim, na ordem dele, pela tela (pela rota, se a feature é só API). Ponha no nome do teste o ID do critério que só esse caminho prova; no Laravel, "E2E" no nome do arquivo (o --filter=E2E e o confere-e2e.py acham o teste por ele). A pré-condição nasce no próprio teste e serviço de fora fica fakeado, como na 4.2. Feature que ainda não existe: pendente com o corpo escrito (->todo(), test.skip, @pytest.mark.skip). Um teste só; não mexa em teste que já existe. Rode o comando da Prova inteiro no fim e me mostre o comando e a saída.
```

modelos/pest-browser/CadastroE2ETest.php · o AC-03 da 4.3 — anotado · não copie do PDF

```
it('AC-03 · mostra "Este e-mail já está cadastrado" abaixo do campo, no navegador', function () {
    User::factory()->create(['email' => 'ana@exemplo.com']);   // pré-condição no teste
    visit('/register')                                          // o caminho do usuário
        ->type('email', 'Ana@Exemplo.com')                      // (nome e senhas: no arquivo)
        ->press('Create account')                              // rótulo do starter kit: troque
        ->assertSee('Este e-mail já está cadastrado')           // o que o usuário vê
        ->assertNoSmoke();                                      // sem erro de JavaScript
    expect(User::count())->toBe(1);                             // e o que ficou no banco
})->todo();                                                     // feature nova: pendente
```

8 **Veja falhar pelo motivo certo e devolva o marcador.**

Feature nova: tire o marcador e rode só o E2E. Ele falha porque o que o critério pede não aparece (rota, seletor ou ambiente quebrado não contam). Devolva o marcador e rode a Prova inteira: verde, com o pendente no placar. Passou sem o marcador? Já existe: fica ativo. Sem Sail: `./vendor/bin/pest --filter=E2E`.

Terminal · projeto Laravel com Sail — laravel

```
vendor/bin/sail test --filter=E2E
vendor/bin/sail test
```

Terminal · projeto Node — node

```
npx playwright test
npm test
```

Terminal · projeto Python — python

```
pytest tests/e2e
pytest
```

9 Proteja e commite.

Com a trava da 4.2 (`guarda-aceite.py`), ponha o caminho do E2E no `.claude/aceite.txt`: o loop só vai poder tirar o marcador. Confira de novo, olhe o `git status` e commite; o loop do Módulo 5 roda a Prova em toda fase.

Terminal · na raiz do seu projeto — shell

```
python3 .claude/scripts/confere-e2e.py
git status --short
git add -A
git commit -m "test: E2E do fluxo principal da feature-alvo"
```

4 Arquivos que *reforçam a aula*

`.claude/scripts/confere-e2e.py`

"Roda no comando da Prova" virou checagem: `phpunit.xml`, `Pest.php`, script `test`, Vitest, navegadores do config, `testpaths`, o app no `conftest.py`. Aceita `--test-cmd`, com o mesmo comando do `LOOP_TEST_CMD`.

No repositório: aula-4.4-e2e-minimo/projeto/

`modelos/pest-browser/SmokeTest.php`

O teste que prova o ambiente: ativo, só erro de JavaScript. Os de Node e Python estão em `playwright/` e `pytest-playwright/`, este com o `conftest.py` que sobe o app.

No repositório: aula-4.4-e2e-minimo/modelos/

`modelos/pest-browser/CadastroE2ETest.php`

O AC-03 do slide, também em Playwright e pytest-playwright. Rodados: pendente, vermelho sem o marcador e verde no controle.

No repositório: aula-4.4-e2e-minimo/modelos/

`modelos/laravel-http/CadastroApiE2ETest.php`

O E2E sem tela da aba da API: rota, 422, erro no campo e banco. O `CadastroHttpE2ETest.php` segue o redirect e lê o HTML.

No repositório: aula-4.4-e2e-minimo/modelos/laravel-http/

`modelos/playwright/vitest.config.ts`

O `exclude` que põe Vitest e Playwright no mesmo `npm test`. Ao lado, o `playwright.config.ts` com `testDir` e `webServer`.

No repositório: aula-4.4-e2e-minimo/modelos/playwright/

`modelos/pedir-o-ambiente.md`

O pedido do passo 5 por inteiro, com o que conferir na resposta. O do passo 7 é o `pedir-o-e2e.md`, ao lado.

No repositório: aula-4.4-e2e-minimo/modelos/

5 Erros *comuns*

SINTOMA	CAUSA	CORREÇÃO
Suíte verde sem o E2E no placar; com `--filter`, "No tests found."	`tests/Browser` fora das `<testsuite>` do `phpunit.xml`	Uma `<testsuite>` com `tests/Browser`
`Target class [config] does not exist.`	`tests/Browser` fora do `->in()` do `tests/Pest.php`	`->in('Feature', 'Browser')`

SINTOMA	CAUSA	CORREÇÃO
Composer: `requires php ^8.4` **ou** `conflicts with ... (^3.8)`	Pest browser pede Pest 4+ e PHP 8.3+	Atualize o Pest; ou HTTP test, ou Dusk (passo 3)
`Playwright is outdated` **(Pest),** `Executable doesn't exist`	Falta o navegador daquela versão; firefox ou webkit: config do `npm init`	`npx playwright install` (Python: `playwright install`) no ambiente da Prova; um navegador no config
`net::ERR_CONNECTION_REFUSED` **(Python)**	O pytest-playwright não sobe o app	Fixture no `conftest.py` (modelo) ou `live_server`
Prova verde sem navegador instalado	Só há E2E pendente: ele não abre o navegador	Smoke ativo (passo 4)
`test.skip() can only be called inside test` **ou** `Vitest failed to find the runner`	Vitest e Playwright pegando o arquivo um do outro	`exclude` no Vitest e `testDir` no Playwright (modelos)
Playwright reprova por "Failed to load resource ... 422"	Conferiu o console inteiro: o 422 da validação vira erro de console	Confira só `pageerror`
Terminal parado depois de `pest | tail`	O Pest browser deixa um `playwright run-server` vivo, e o pipe espera	Sem pipe, ou grave em arquivo
`--debug` **não abre janela**	Dentro do container ou no CI não há tela	Fora do container; dentro, leia o screenshot da falha

6 Você sai *com*

- [ ] `confere-e2e.py` dizendo "os E2E entram no comando do juiz", e o smoke passando na Prova (Node e Python: com o app fechado).
- [ ] Um E2E do fluxo principal da feature-alvo com o ID do critério no nome. Feature nova: falhou pelo motivo certo e voltou para pendente.
- [ ] O comando da Prova inteiro verde, com o pendente no placar, e o E2E no commit.
- [ ] O Módulo 4 completo: Prova e juiz (4.1), contrato (4.2), critérios com ID e um teste por critério (4.3), E2E do fluxo (4.4).

PROMETIDO NA AULA

- ✓ **"No Laravel, o Pest navegador é dessa forma aqui"**: `modelos/pest-browser/CadastroE2ETest.php`, lido no passo 7 (p. 7); a instalação no passo 3 (p. 5).
- ✓ **"Para Node, Python, tem os pacotes específicos do Playwright"**: passo 3 (p. 5), `modelos/playwright/` e `modelos/pytest-playwright/`.

- ✓ **"Sem tela, a gente consegue fazer isso via HTTP"**: `modelos/laravel-http/CadastroApiE2ETest.php` (a aba da API) e `CadastroHttpE2ETest.php`, na p. 8.
- ✓ **"O fluxo principal de uma feature, o serviço de fora fakeado"**: o pedido do passo 7 (p. 7, `modelos/pedir-o-e2e.md`) e o gap "onde o dublê vive" (p. 4). O gate da Prova ("a gente vai construir esse gate depois") é a aula 5.1.
- ✓ **"O navegador interno precisa estar instalado dentro desse container"**, **"ou ao menos solicitar ao seu assistente de código"**: o smoke (passo 4) e `modelos/pedir-o-ambiente.md` (passo 5, p. 6).
- ✓ **"Com os testes configurados, a gente consegue rodar na nossa stack de testes"**: passos 6 e 8 (p. 6 e 7), no comando da Prova, com o `confere-e2e.py`.

7 *Glossário*

E2E (ponta a ponta)	Teste que usa a feature como o usuário, da tela (ou da rota) até o resultado.
Smoke	Teste curto e sempre ativo que prova que o comando da Prova abre um navegador.
Prova	O comando de teste que o loop roda, fora da sessão do agente, e que volta passa ou falha (4.1). No `loop.sh`, vai no `LOOP_TEST_CMD`.
Pendente	Teste com o corpo escrito e um marcador que o pula: `->todo()`, `test.skip`, `@pytest.mark.skip`.
Pest browser	Plugin do Pest que testa no navegador com o Playwright por baixo: `visit()`, `type()`, `press()`, `assertSee()`.
assertNoSmoke()	Asserção do Pest browser: sem log no console nem erro de JavaScript na página.
webServer	Opção do `playwright.config` (Node) que sobe o app e espera a URL responder. No Python, é uma fixture do `conftest.py`.
Fake (dublê)	Serviço de fora trocado no teste por uma versão que respeita o contrato (4.2).

8 Para *ir além*

- Pest · Browser Testing: instalação, asserções, `--debug`, `--trace` e o trecho de CI.
- Laravel · HTTP Tests: `postJson`, `followingRedirects` e a requisição simulada.
- Playwright · Web server: `webServer`, `reuseExistingServer` e `env` para subir o app no teste.
- Playwright · Browsers: o Chromium de cada versão e o `install-deps`.
- Anthropic · Effective harnesses for long-running agents: o agente que marcava feature pronta sem testar o fluxo e melhorou testando "as a human user would".

Próxima: *Aula 5.1 · O script do loop*

Agora o loop tem Prova, e o E2E entra nela. Na 5.1 a feature vai para o loop: cada fase passa pelo mesmo comando, fora da sessão do agente.